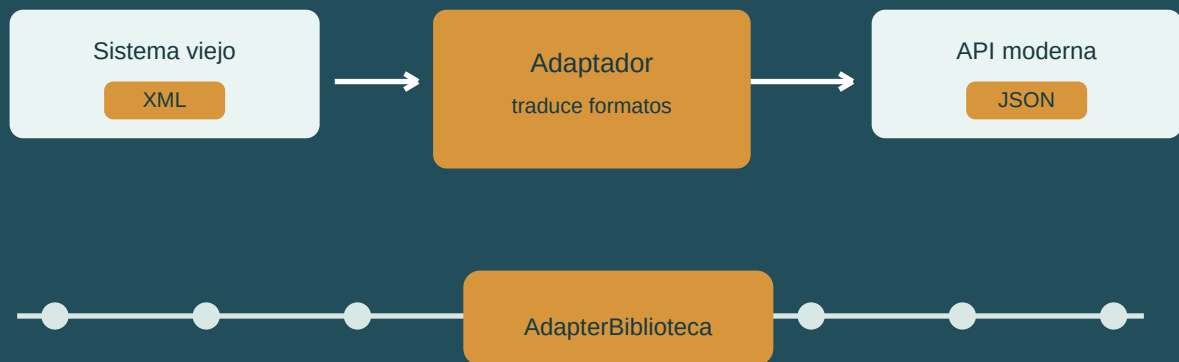


Patron Adapter

Guia visual del proyecto patron-adapter: como una biblioteca vieja que habla XML se conecta con una API REST moderna.

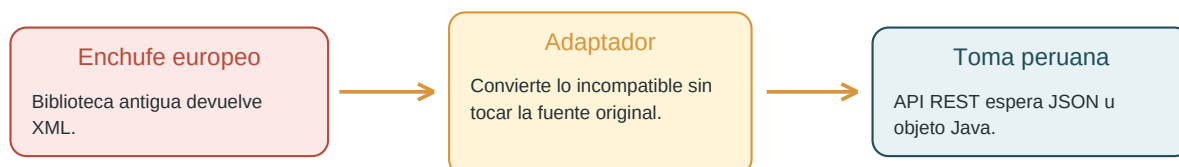


Caso de estudio: Biblioteca, AdapterBiblioteca, BibliotecaController y el modelo Libro.

1. La Idea En Una Frase

Adapter es un patron estructural: permite que dos piezas incompatibles trabajen juntas sin modificar la pieza antigua. En este proyecto, la pieza antigua es una biblioteca que devuelve XML; la API nueva quiere JSON u objetos Java.

Analogia: tienes un cargador europeo y una toma peruana. No rompes el cargador ni cambias la pared. Pones un adaptador entre ambos. Ese adaptador no inventa energia: solo traduce la forma de conexion.



Adapter no cambia el aparato viejo; solo hace que encaje con el sistema nuevo.

Imagen 1. El adapter funciona como un puente entre una salida vieja y una entrada nueva.

2. Quienes Son Los Personajes Del Proyecto

Clase	Rol en el patron	Que hace
Biblioteca	Sistema antiguo / Adaptee	Devuelve informacion de libros en XML como String.
AdapterBiblioteca	Adapter	Llama al sistema antiguo y convierte XML a JSON o a Libro.
BibliotecaController	Cliente moderno	Expone endpoints REST y no se mete con la traduccion.
Libro, Editorial, Autor...	Modelo nuevo	Representan los datos ya entendibles para Java/Spring.

3. La Clase Vieja: Biblioteca

La clase antigua simula un sistema que ya existe. Su contrato es simple: le das un id y te devuelve texto XML. El problema no es que este mal; el problema es que no encaja con lo que espera tu API REST moderna.

```
public String obtenerInfo(int idLibro) {  
    return "<libro> ... </libro>";  
}
```

Si tu controller consumiera esto directamente, tendria que saber demasiado sobre XML. Eso mezclaria responsabilidades.

4. El Adapter: Donde Ocurre La Traduccion

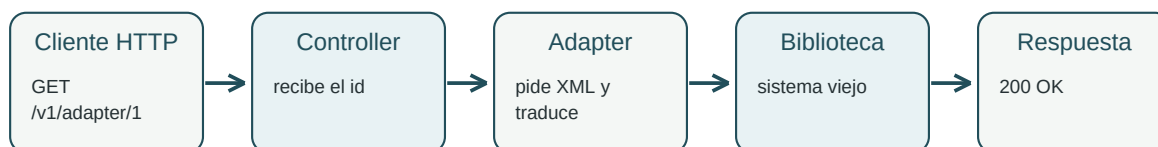
AdapterBiblioteca es el traductor. Por dentro usa Biblioteca, pero por fuera ofrece metodos mas amigables para el controller.

```
@Component
public class AdapterBiblioteca {
    private final Biblioteca biblioteca = new Biblioteca();
    private final XmlMapper xmlMapper = new XmlMapper();

    public String obtenerDetalle(int idLibro) {
        String detalle = biblioteca.obtenerInfo(idLibro);
        JSONObject jsonObject = XML.toJSONObject(detalle);
        return jsonObject.toString();
    }
}
```

Piensalo como un interprete: escucha XML, responde JSON. El controller no aprende XML; solo llama al interprete.

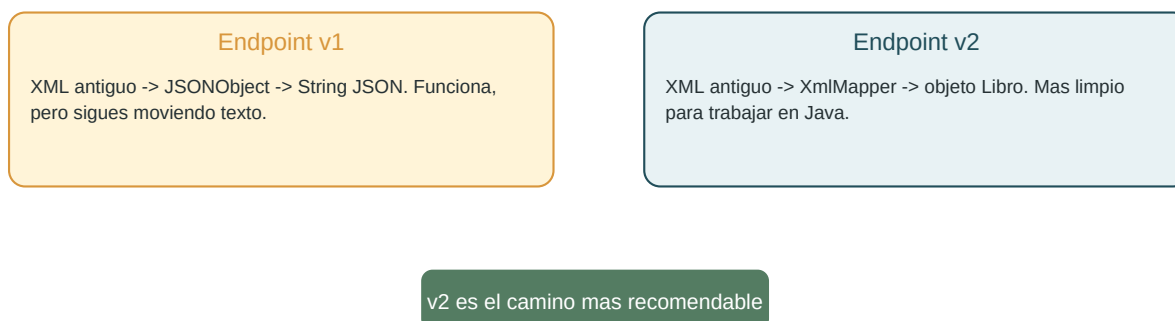
5. Flujo Completo



El controller no sabe traducir XML: delega ese trabajo al adapter.

Imagen 2. El request entra por el controller, pero la conversion vive en AdapterBiblioteca.

6. Los Dos Endpoints



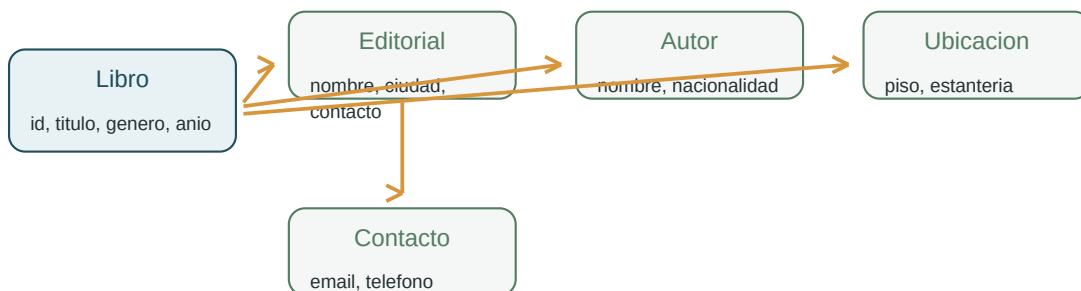
El proyecto tiene dos endpoints parecidos, pero no hacen exactamente lo mismo.

Endpoint	Devuelve	Como adapta
----------	----------	-------------

GET /v1/adapter/{id}	ResponseEntity	Convierte XML a JSON como texto usando XML.toJSONObject.
GET /v1/adapter/v2/{id}	ResponseEntity	Convierte XML a objeto Libro usando XmlMapper.

La version v2 suele ser mas limpia, porque ya trabajas con un objeto Java. Luego Spring se encarga de convertir ese objeto a JSON al responder.

7. Por Que Libro Se Llena Solo



Las etiquetas del XML calzan con campos Java, por eso Jackson puede llenar el objeto.

Imagen 3. La estructura del XML se parece a la estructura de las clases Java. Por eso Jackson puede mapear etiquetas XML a campos.

```

public class Libro {
    private int id;
    private String titulo;
    private String genero;
    private int anio;
    private Editorial editorial;
    private List<Autor> autores;
    private Ubicacion ubicacion;
}
  
```

Cuando el XML trae , Jackson busca un campo llamado editorial. Cuando trae , busca ubicacion. El adapter prepara la traduccion y el modelo recibe los datos.

8. Por Que No Usar Biblioteca Directo En El Controller

- Porque el controller deberia encargarse de HTTP, no de traducir XML.
- Porque si manana la biblioteca vieja cambia su formato, solo tocas el adapter.
- Porque mantienes aislado el codigo antiguo.
- Porque la API moderna queda limpia: pide datos y recibe datos adaptados.

Otra analogia: un restaurante. El cliente no entra a la cocina a explicar cada paso. Habla con el mesero. El mesero traduce el pedido al lenguaje de cocina y luego trae el plato listo.

9. Lectura Rapida DelCodigo

```

@GetMapping(value = "/v2/{id}")
public ResponseEntity<Libro> obtenerLibro2(@PathVariable Integer id)
    throws JsonProcessingException {
  
```

```
    return ResponseEntity.ok(biblioteca.obtenerDetalle2(id));  
}
```

Lectura humana: cuando llegue una peticion a /v2/1, toma el id, pide al adapter que obtenga el libro adaptado y responde 200 OK con ese Libro.

10. Que Debes Recordar

- Adapter no reemplaza al sistema viejo: lo envuelve.
- Adapter traduce una interfaz/formato incompatible a uno compatible.
- En este proyecto, Biblioteca habla XML y AdapterBiblioteca traduce.
- El controller queda limpio porque no sabe como se hace la conversion.
- La version v2 es mas orientada a Java porque devuelve Libro, no texto.

Frase final: Adapter es el puente entre lo que ya tienes y lo que tu aplicacion necesita hoy.

11. Referencias Para Profundizar

Refactoring.Guru - Adapter

<https://refactoring.guru/design-patterns/adapter>

Explicacion visual del patron adapter, sus roles y casos de uso.

Spring Framework - ResponseEntity

<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/http/ResponseEntity.html>

Documentacion oficial de la clase usada por los controllers para devolver respuestas HTTP.

Jackson XmlMapper

<https://github.com/FasterXML/jackson-dataformat-xml>

Libreria usada para convertir XML a objetos Java.